

Inference: Disaggregation and Fusion

1. MPK fuses GPU work into a persistent kernel
2. Mooncake separates serving stages and reuses KV state

Angela Li, Jack McClure, Saitej Velaga

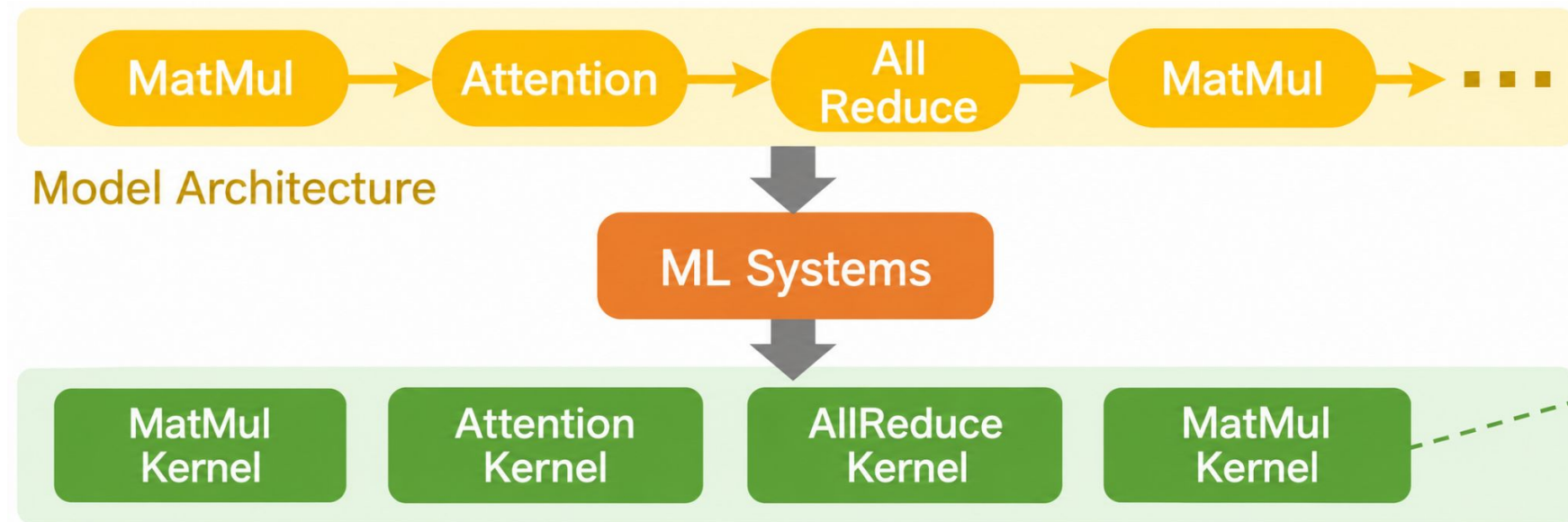
MPK: A Compiler and Runtime for Mega-Kernelizing Tensor Programs

- Traditional compilation and runtime
- MPK - Mirage Persistent Kernel

Agenda:

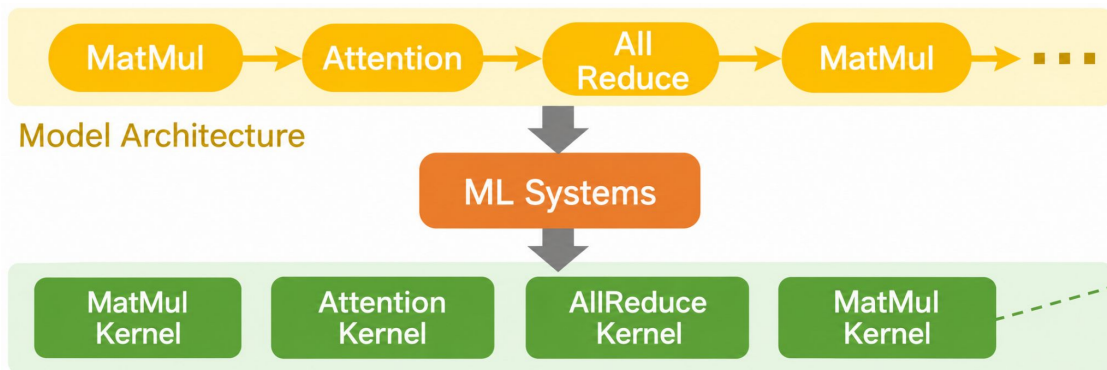
1. Explain how separate kernels create launch overhead, pipeline bubbles, lost overlap
2. Introduce persistent mega-kernels and why compiler is needed to build them
3. Show how the task graph and GPU workers/schedulers coordinate ready work
4. Summarize MPK's MoE optimizations (fused gather GEMM and dynamic load balancing)
5. Evaluate launch, pipelining, and multi-GPU performance gains

Traditional Compilation and Runtime for LLM Inference



Traditional compilation and runtime for LLM Inference - Benefits

- Each operation is implemented and optimized independently
- Easy data dependency
 - One wave must complete before another can start



Traditional compilation and runtime for LLM Inference - Negatives

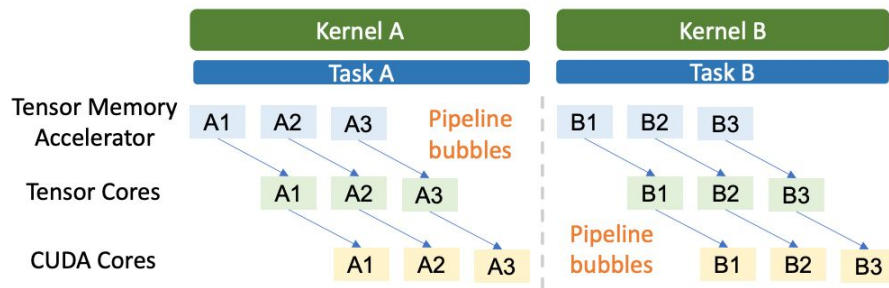
- Lost overlap opportunity
 - Communication and computation overlap is limited
- Ready work cannot begin until prior kernel fully completed
 - Overhead of launching kernels



(a) Kernel barriers prevent overlapping MatMul and AllGather.

Pipeline Bubbles

- Traditional inference leads to bubble formation as not all the hardware is used
- This is due to the kernel boundaries
- Ready work of B cannot use any of the open processing available



(a) Kernel barriers prevent cross-task pipelining.

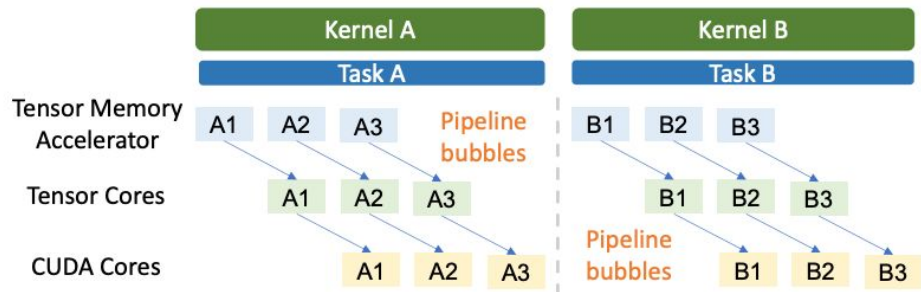
Lack of Overlap

- MatMuls are bound by compute
- AllGather have communication overhead
- Can't overlap communication with compute

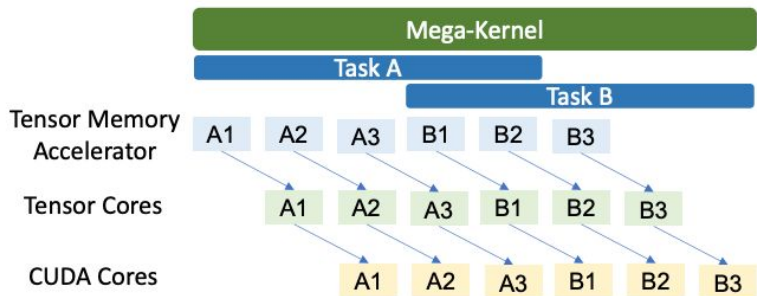


(a) Kernel barriers prevent overlapping MatMul and AllGather.

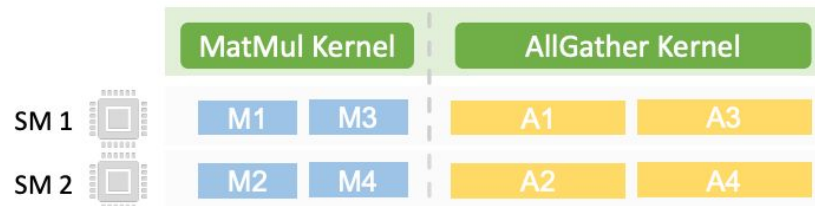
MPK - Mirage Persistent Kernel



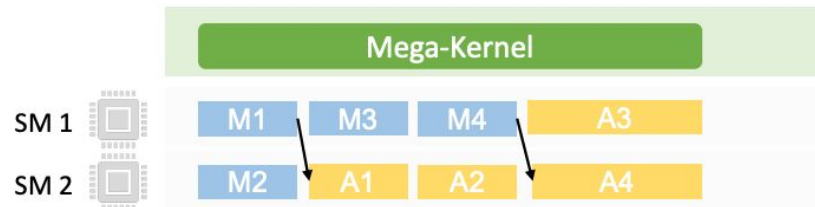
(a) Kernel barriers prevent cross-task pipelining.



(b) MPK enables both intra- and cross-task pipelining.



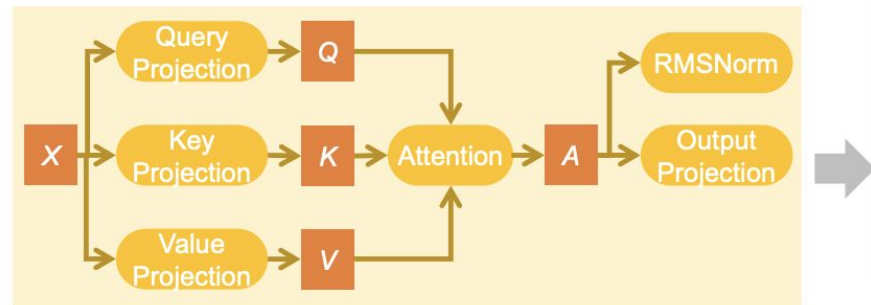
(a) Kernel barriers prevent overlapping MatMul and AllGather.



(b) MPK enables fine-grained overlap of MatMul and AllGather.

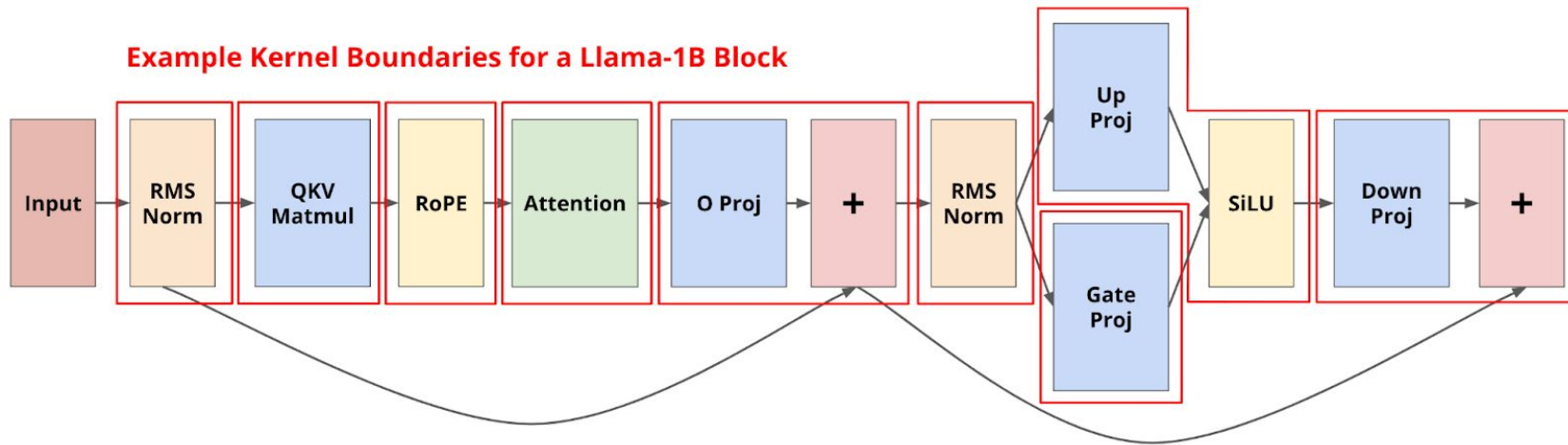
Kernel Types

- Traditional: launch a new GPU kernel for each operation
- Persistent kernel: one kernel stays running on the GPU and executes many tasks internally
- Mega-kernel: combines many operators
- fewer kernel boundaries and more opportunity to overlap ready work



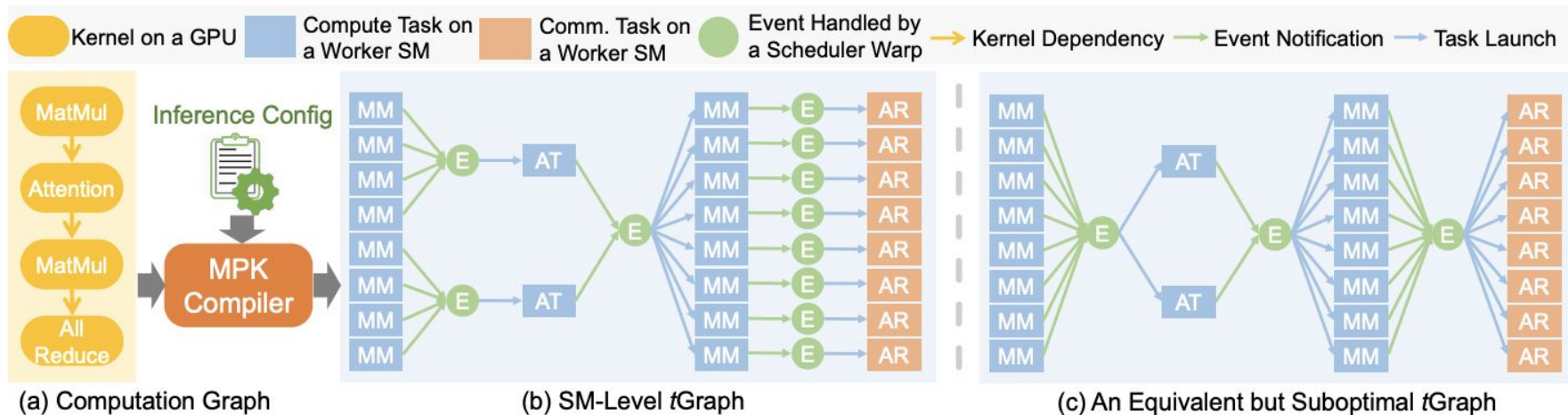
MPK - Why Build a Compiler

Example Kernel Boundaries for a Llama-1B Block



- It is difficult to write a mega kernel by hand
 - Expensive
 - Time consuming
- Changing a model should not require a rewrite of a complicated mega kernel

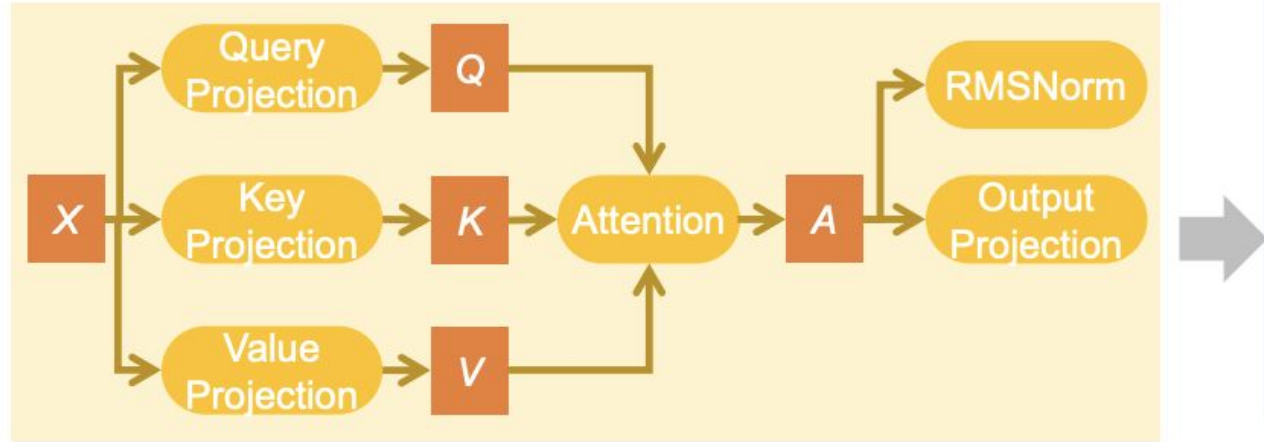
MPK - Mirage Persistent Kernel



- Combine the whole model into one kernel
- Everything handled by the kernel
 - Communication between GPUs
 - Data dependency

MPK - Computation Graph

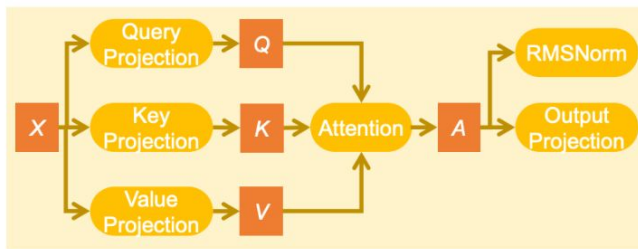
- Start with the original computation graph for inference



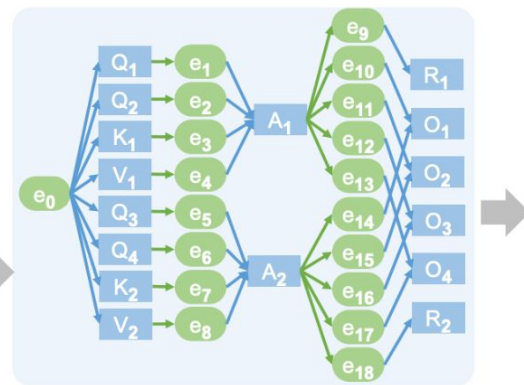
(a) Computation Graph

MPK - Operator Decomposition and Dependency Analysis

- Break up large operations (attention, RMSNorm) into smaller GPU tasks



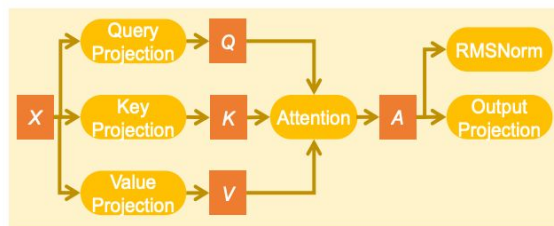
(a) Computation Graph



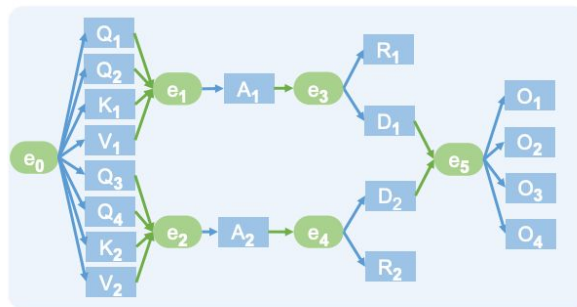
(b) Operator decomposition & dependency analysis

MPK - tGraph Normalization/Linearization

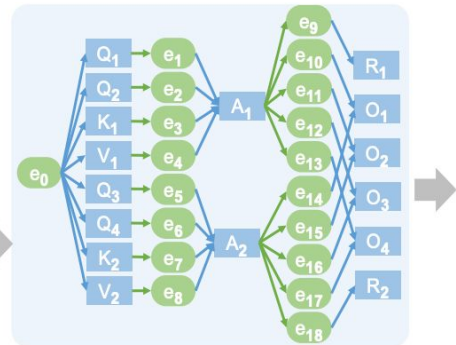
- Simplify the tGraph by merging redundant events and making a task only trigger one event



Computation Graph



tGraph normalization

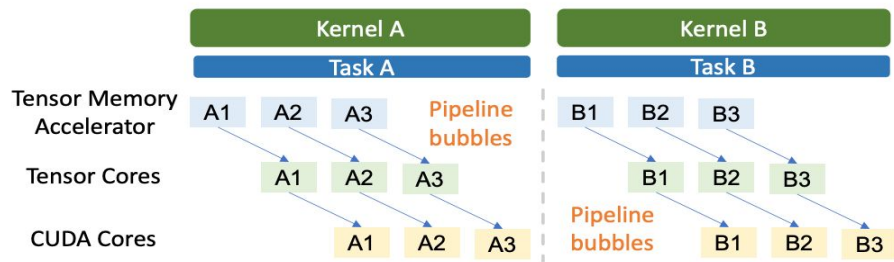


Operator decomposition & dependency analysis

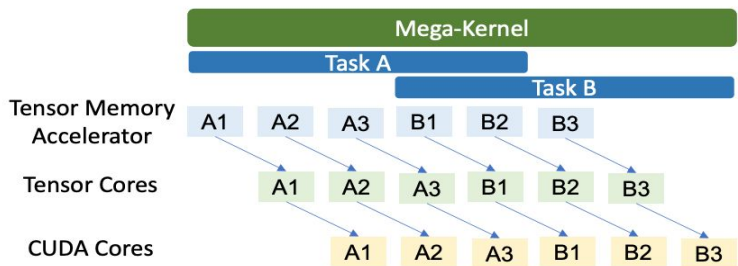
IDs	1	2	3	4	5	6	7	8	9
Tasks	Q ₁	Q ₂	K ₁	V ₁	Q ₃	Q ₄	K ₂	V ₂	A ₁
Depend	e ₀	e ₀	e ₀	e ₀	e ₀	e ₀	e ₀	e ₀	e ₁
Trigger	e ₁	e ₁	e ₁	e ₁	e ₂	e ₂	e ₂	e ₂	e ₃
IDs	10	11	12	13	14	15	16	17	18
Tasks	A ₂	R ₁	D ₁	R ₂	D ₂	O ₁	O ₂	O ₃	O ₄
Depend	e ₂	e ₃	e ₃	e ₄	e ₄	e ₅	e ₅	e ₅	e ₅
Trigger	e ₄	-	e ₅	-	e ₅	-	-	-	-
Events	e ₀	e ₁	e ₂	e ₃	e ₄	e ₅			
Counts	0	4	4	1	2	1			
First task ID	1	9	10	11	13	15			
Last task ID	8	9	10	12	14	18			

tGraph linearization

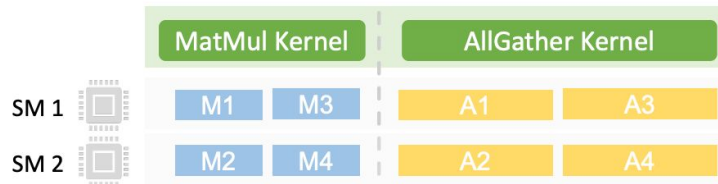
Results: MPK Removes Barriers to Cross-Task Pipelining



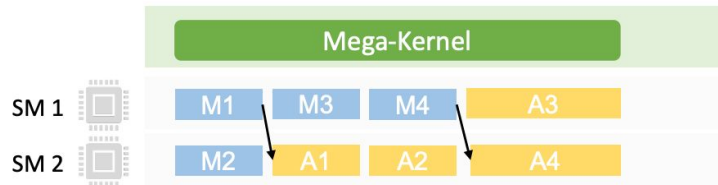
(a) Kernel barriers prevent cross-task pipelining.



(b) MPK enables both intra- and cross-task pipelining.

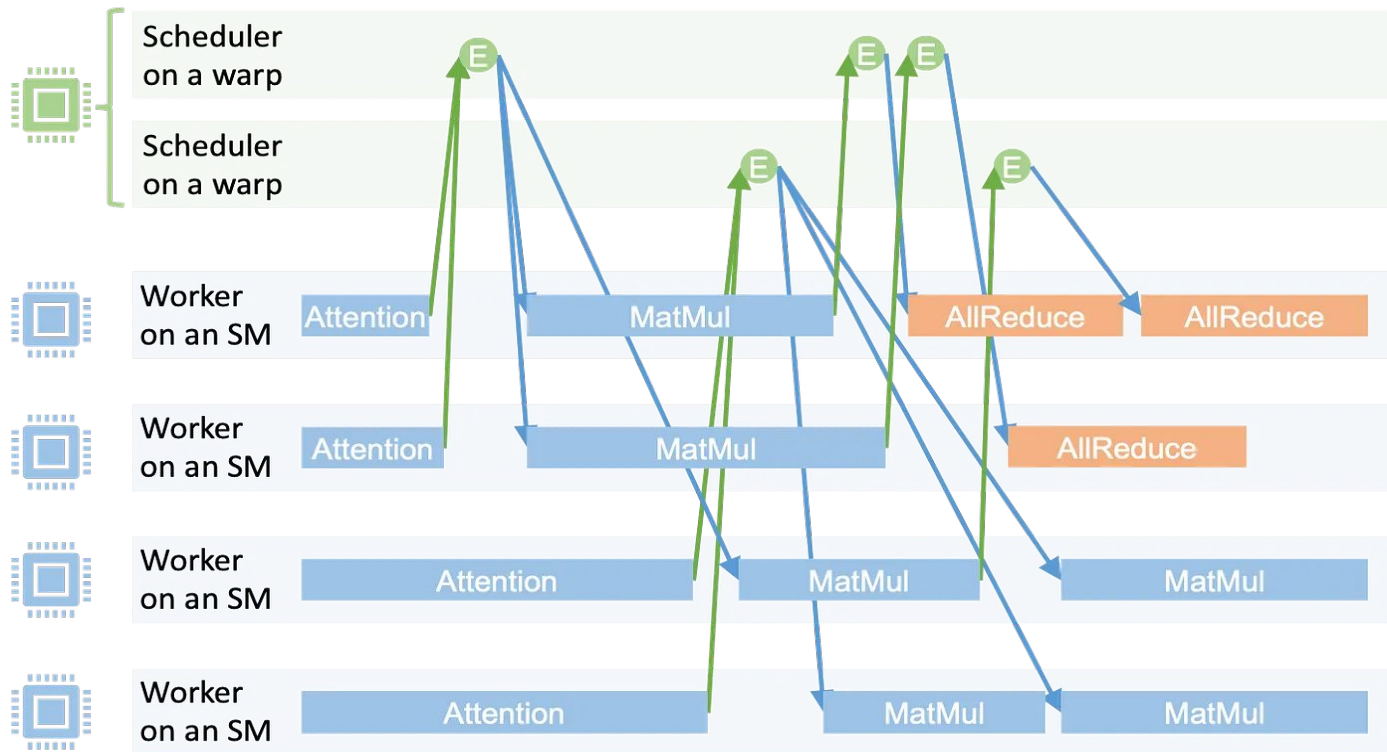


(a) Kernel barriers prevent overlapping MatMul and AllGather.



(b) MPK enables fine-grained overlap of MatMul and AllGather.

GPU Implementation



GPU Implementation

- GPUs have SMs (streaming multiprocessor)
- We assign these SMs to be either workers or schedulers
- Schedulers assign tasks to worker queue
 - Tasks are assigned at either
 - runtime (JIT)
 - compile time (AOT)
- AOT tasks have predictable runtime
 - JIT tasks may have unpredictable runtime
 - Attention is unpredictable as sequence length impacts runtime

Table 1: MPK configuration in our evaluation.

GPU	# SMs	# Workers	# Schedulers
A100	108	104	16
H100	132	128	16
B200	148	144	16

One launch saves overhead; pipelining improves runtime

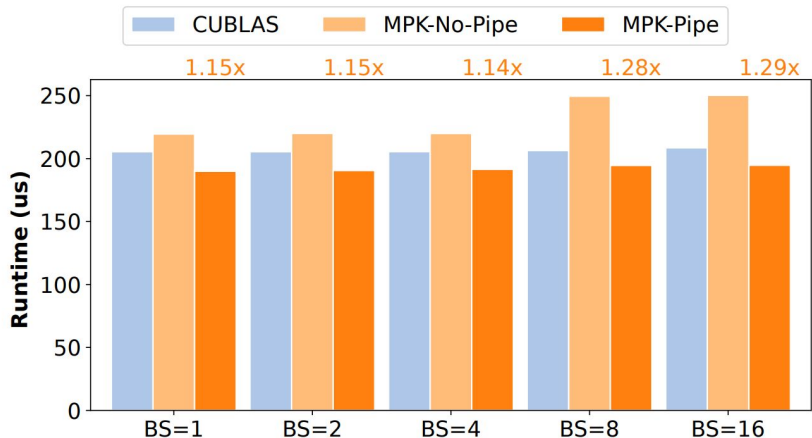
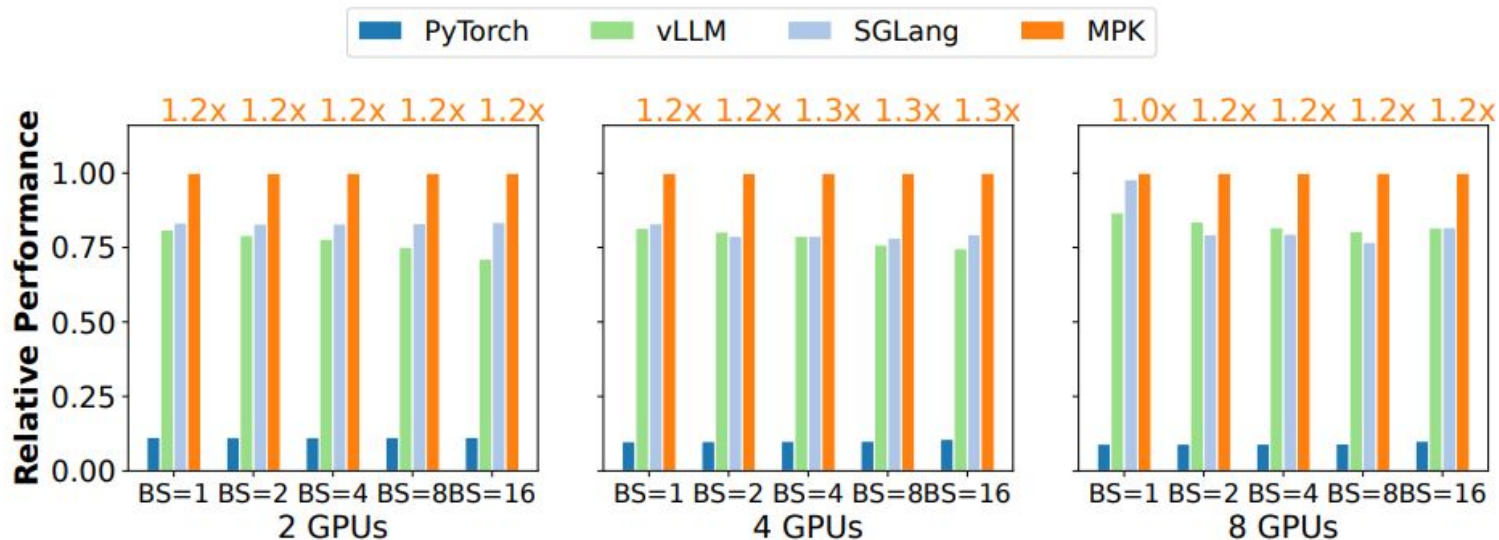


Figure 12: Ablation study of cross-task pipelining. We measure the runtime of the final linear layer in Qwen3-8B on an NVIDIA B200 GPU and report execution time in microseconds; lower is better. The value above each bar reports the speedup of MPK-Pipe over MPK-No-Pipe.

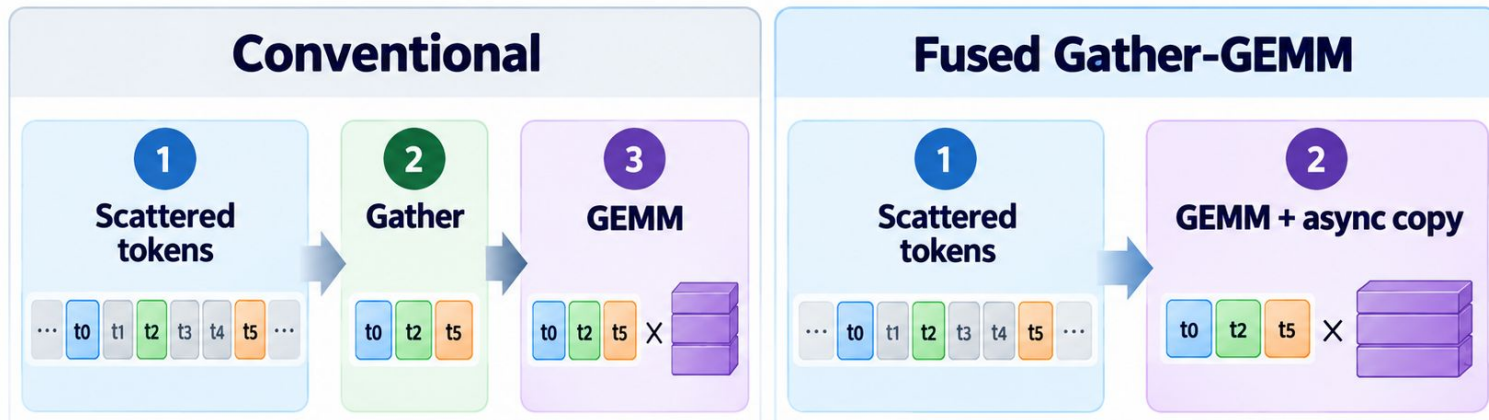
- **293 Qwen3-8B operator launches per token become one MPK launch.**
- B200: 1.1 ms eager or 0.2 ms CUDA Graph launch overhead; pipelining gains 1.14–1.29x.

Results: MPK Retains Speedups Across Multi-GPU Inference



- Qwen3-1.7B on H100 tensor parallelism
- At 8 GPUs: 1.1-1.4x faster than SGLang and vLLM
 - Across single-batch tests: 1.0-1.7x speedup
 - Values in the chart are normalize to MPK

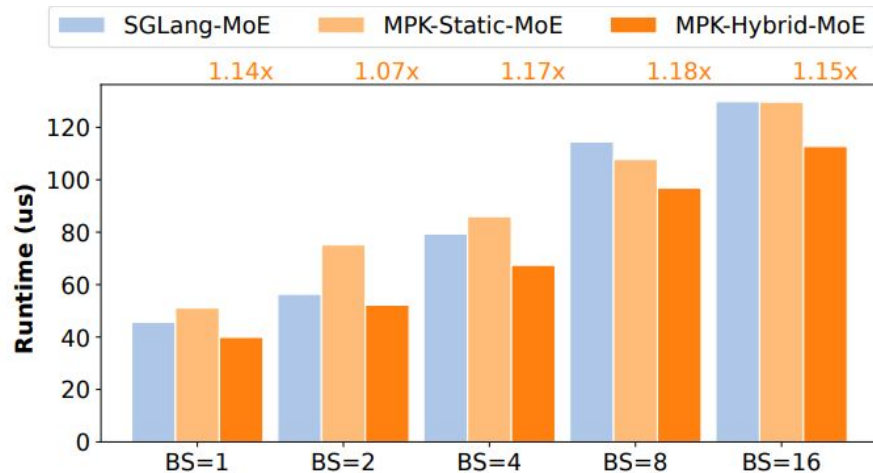
MoE - Fused Gather GEMM



- In traditional MoE inference you need to gather tokens for the same expert in contiguous memory before executing
- In Fused Gather GEMM, the gather step is done asynchronous
 - GEMM operations are being performed for other pieces of data

MOE - Hybrid Workload Balancer

- Some experts are more popular than others
 - The load balance is unknown until runtime
- MoE computation are partitioned into expert specific tasks
- Each tasks receives a meta tensor on information about expert usage
- Meta tensor information is used to dynamically split work evenly across SMs so all experts will finish at the same time



A faster pass can still repeat the same work

- MPK: reduces waiting within a model pass
- Mooncake: avoids repeated work on shared prefixes

The resource boundary changes: GPU scheduling to reuse across requests

Agenda:

1. Introduce prefill/decode, latency targets, why long prompts create interference
2. Explain prefix KV reuse, why local caches leave reusable work scattered across cluster
3. Compare local caching, chunked prefill, prefill/decode separation to identify what remains unsolved
4. Follow a request through Mooncake Store, transfer engine, Conductor

Mooncake: motivation, design, evidence

Motivation

- Long prompts delay responses, and local caches miss reusable prefixes

Design

- Mooncake shares KV state and routes requests to use it

Evaluation

- The results measure capacity, saved prefill work, and network cost

Prefill and decode create competing latency demands

Prefill is compute bound

- A longer prompt delays the first token

Decode is memory bound

- Each token should arrive at a steady pace

The serving goal must respect both first-token (TTFT) and between token (TBT) limits

The same opening tokens produce reusable KV state

Request 1

Same long document + instructions

Question A

Prefill creates and stores KV for the shared prefix.

Request 2

Same long document + instructions

Question B

Request 2 loads the saved KV and computes only its new question

What existing approaches solve, and miss

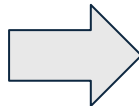
Approach	Helps with	Still left
Local prefix cache	Avoids recomputing local hits	A different prefix node may provide a cache hit instead
Chunked prefill	Gives decode turns between chunks	Prefixes cached on one prefill node may still go unused on others
P/D separation	Protects decode from long prefills	Reusable KV cached on prefill nodes may go unused on decode nodes

Local capacity misses dispersed prefixes

One worker cannot keep every useful prefix in its local cache

One local cache

For LLaMA3-70B, about 1 TB holds 3 million KV tokens



The useful working set

Matching prefixes are spread across workers and requests

An exact prefix can exist in the cluster and still be a miss on the chosen worker

The problem Mooncake chooses to solve

Mooncake trades storage and network for less repeated GPU computation

1. Capacity

The cluster retains useful prefixes beyond one local cache

2. Transfer

Remote KV must arrive sooner than recomputation would finish

3. Routing

The chosen worker must meet the latency target after queue and transfer delays

A remote cache hit helps only when fetching it costs less than recomputing

Mooncake connects the previously isolated caches

Capacity

Mooncake Store makes KV blocks accessible across nodes

Movement

The transfer engine fetches cached KV and forwards new KV to decode

Decision

Conductor chooses workers based on hits, queues, transfers, and latency targets

Reuse and move KV state before decoding

A request contains cached prefix S followed by a new suffix Q

Find KV(S)

The prefill worker loads matching state from Mooncake Store.



Prefill Q

The worker computes only tokens after the cached prefix.



Decode

The decoder receives the needed KV and then streams tokens.

- New KV can move while later layers compute
- Decode starts after the needed state arrives

A cache hit is useful only if it wins the race

Worker A · local

It has a short match, a small queue, and no transfer cost.

Worker B · remote

It has a longer match, but the request must wait and transfer KV.

Conductor estimates TTFT from transfer, queue wait, and uncached prefill

- The longest prefix match does not always produce the first token fastest

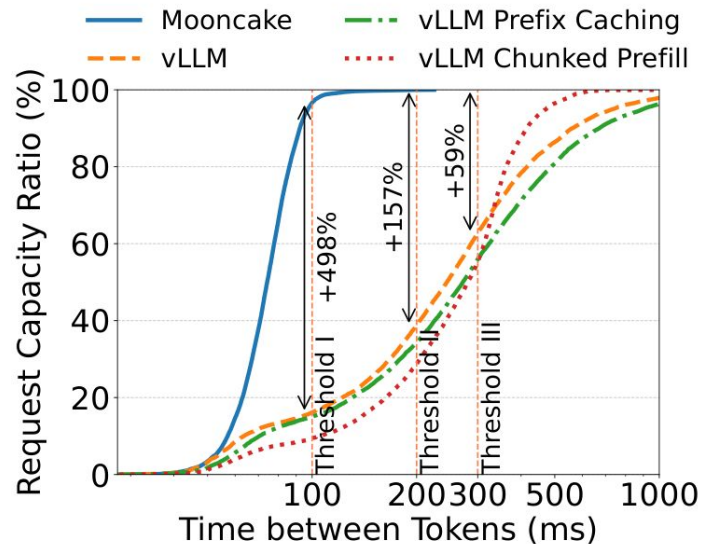
Mooncake evaluation: setup and questions

Testbed	The test uses 16 hosts with 8 A800 GPUs each and a dummy 70B model (based on LLaMA3-70B)
Workloads	It replays Kimi conversation and tool/agent traces plus synthetic requests
Comparison	The baselines are vLLM, local prefix caching, and chunked prefill

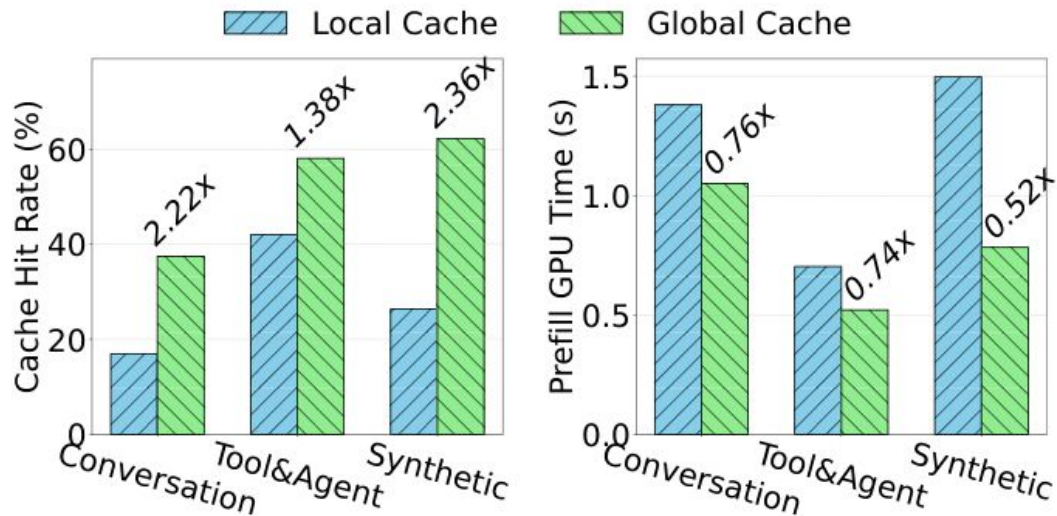
The tests ask whether more requests meet both latency targets and whether global reuse saves prefill work

Strict token timing exposes a capacity gap

- Moving right allows more time between tokens
- Moving up means more requests meet both latency targets
- Mooncake serves more qualifying requests when token timing is strict

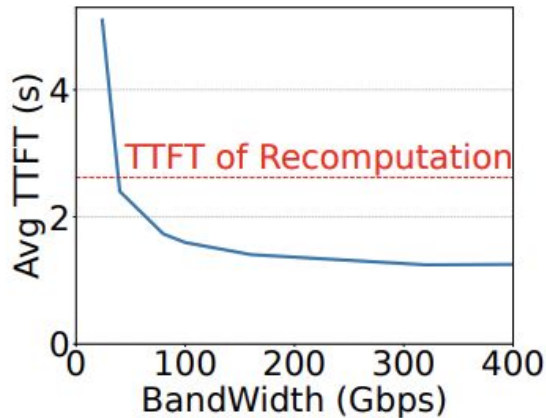


Global reuse cuts repeated prefill work

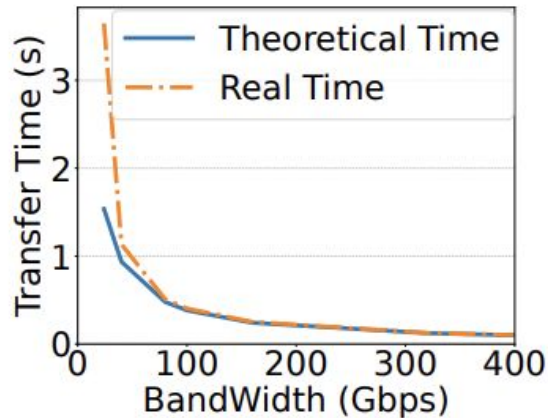


- With the same local cache budget, global sharing raises hit rates and reduces prefill GPU time
- The synthetic trace saves 48%

Network bandwidth sets the break-even point



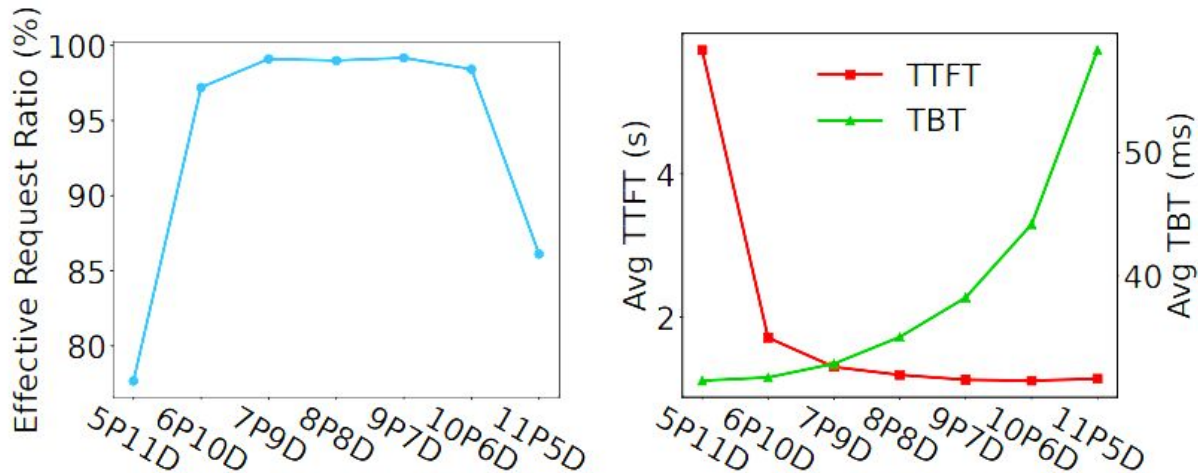
(a) Average TTFT.



(b) Transfer time.

- In the synthetic test, transfer becomes costly below about 100 Gbps
- Remote reuse helps only when fetching the KV state beats recomputing it

Balanced prefill and decode nodes maximize performance



- Mooncake performs best at roughly a 1:1 ratio of prefill to decode nodes, maximizing effective requests while minimizing TTFT and TBT

What the Mooncake results do, and do not, show

The results support the design

- The tested traces show higher effective capacity, and global reuse reduces prefill GPU time

The boundary remains important

- Benefits depend on exact-prefix reuse, cache capacity, bandwidth, and the serving workload

- The evaluation combines caching, scheduling, transfer, and phase separation
- Their separate end-to-end contributions remain an open question

Related work: MPK automates end-to-end mega-kernel fusion

Closest systems and what MPK changes

- **TVM / Triton**

Optimize individual operators or graph regions; MPK compiles the full tensor program and coordinates SM-level tasks inside one persistent kernel.

- **FlashDMoE / hand-built mega-kernels**

Show that aggressive fusion can be fast, but require model-specific GPU engineering; MPK generates the mega-kernel automatically.

- **MPK + Mooncake are complementary**

Mooncake routes requests and KV state across nodes; MPK can accelerate execution inside the chosen prefill/decode workers. This combination is not evaluated in either paper.

Related work: Mooncake makes KV reuse cluster-wide

Closest systems and what Mooncake adds

- **SGLang / RadixAttention**

Reuses common prefixes with a local cache structure; Mooncake pools KVCache across nodes and routes requests using both cache location and load.

- **Splitwise-style prefill/decode disaggregation**

Separates prefill and decode compute; Mooncake additionally disaggregates KV storage/transfer and adds a global cache-aware scheduler.

- **Mooncake + MPK are complementary**

Mooncake reduces cluster-level recomputation; MPK reduces GPU kernel barriers and launch overhead within each inference worker.

Further research

Next directions for both systems

MPK

- **Test real online serving**
Bursty arrivals, variable lengths, and TTFT/TBT SLOs.
- **Adapt tGraphs and scheduling**
Handle dynamic batches and compare decentralized vs. global policies.
- **Broaden portability**
Target more accelerators and deploy MPK inside cluster-level serving systems.

MOONCAKE

- **Adapt cache and P/D decisions**
Tune replication, eviction, and the prefill/decode split as traffic changes.
- **Reduce network dependence**
Use compression, prefetching, and smarter placement on slower networks.
- **Combine Mooncake with MPK**
Mooncake manages cluster-wide KV reuse; MPK can optimize GPU execution inside workers.